Homework 3 — SQL Fundamentals (Single-Table Queries)

SECTION A — ZAGI (5 Queries)

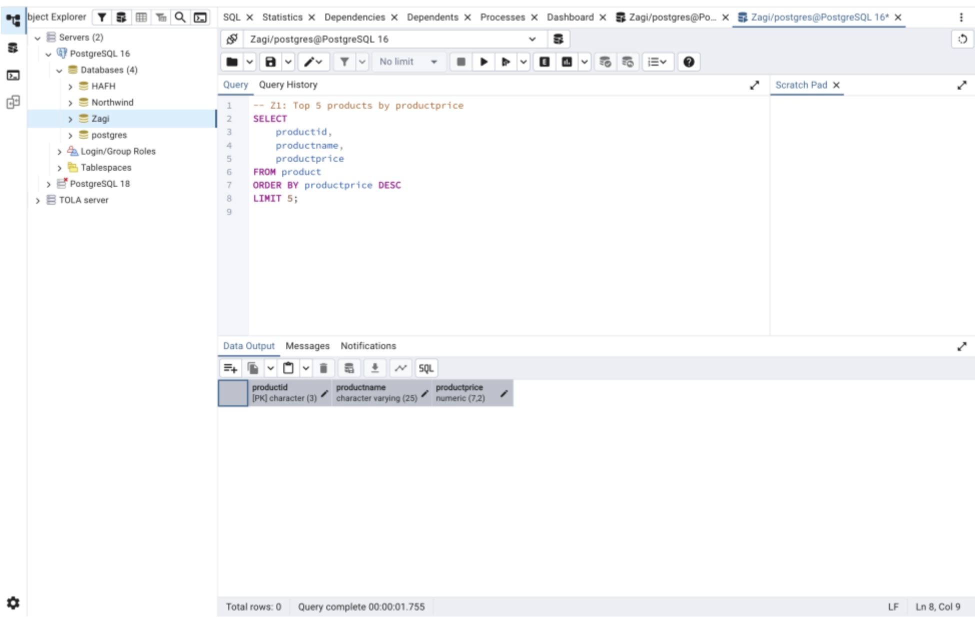
Question:

List the top 5 products by product price (columns: productid, productname, productprice) in descending price order.

SQL CODE:

```
-- Z1: Top 5 products by productprice

SELECT

   productid,

   productname,

   productprice

FROM product

ORDER BY productprice DESC

LIMIT 5;
```

OUTPUT:



Z2 — Categories With More Than 2 Products

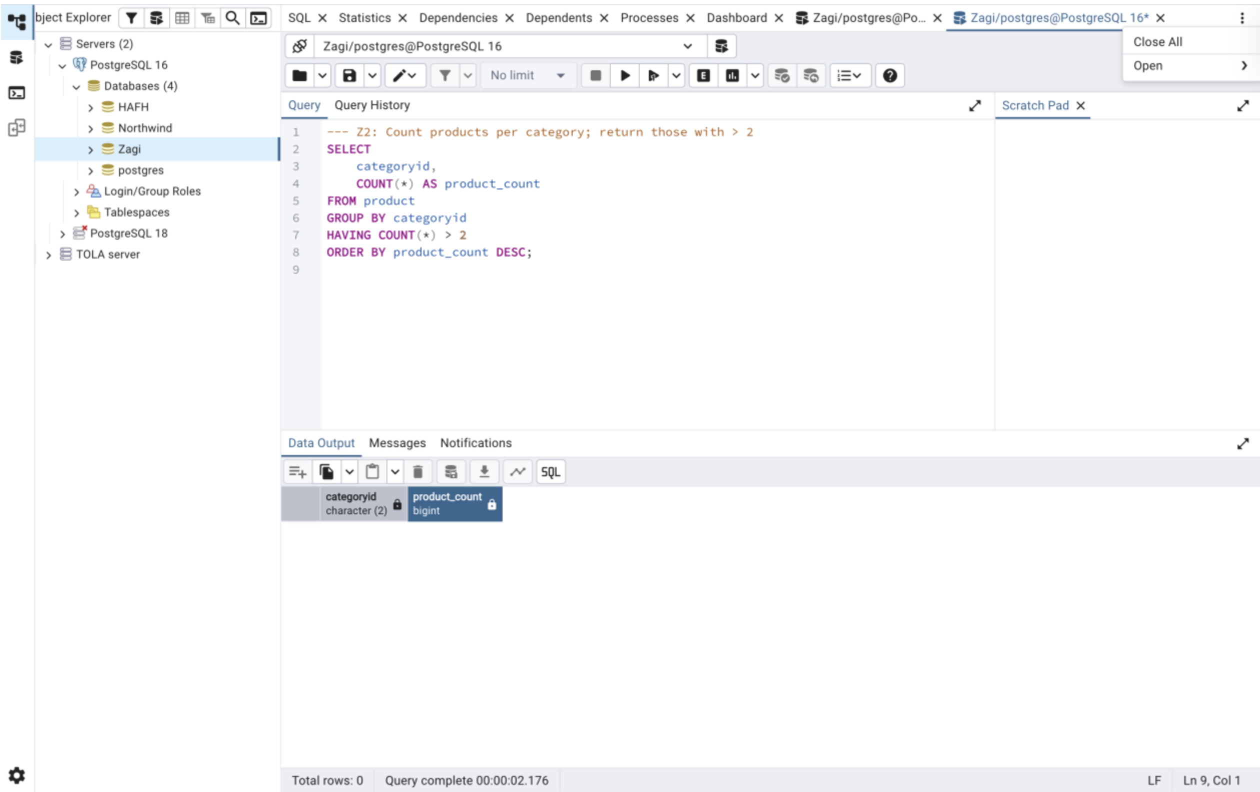
Question:

Using only the product table, return categoryid values with more than 2 products.

SQL Code:

```
-- Z2: Count products per category; return those with > 2

SELECT

    categoryid,

    COUNT(*) AS product_count

FROM product

GROUP BY categoryid

HAVING COUNT(*) > 2

ORDER BY product_count DESC;
```

OUTPUT:



Z3 — Transactions Between 2013-01-02 and 2013-01-05

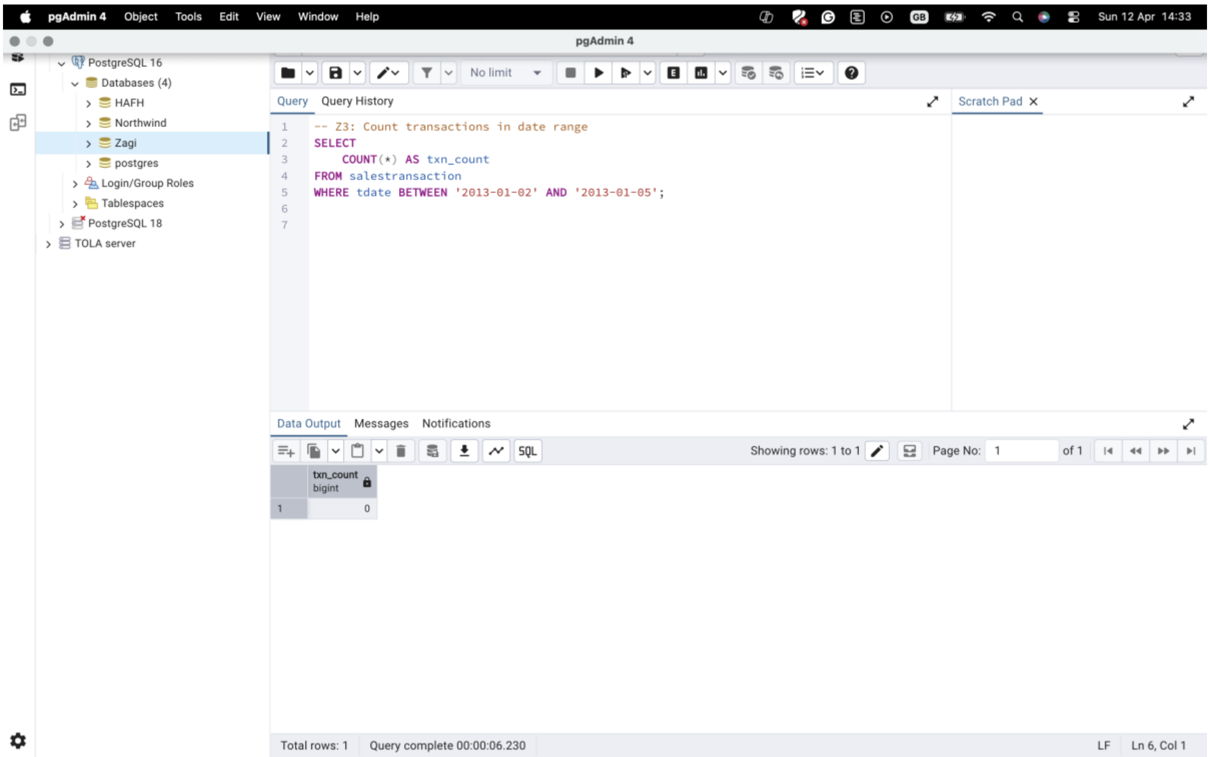
Question:

Count how many transactions occurred between '2013-01-02' and '2013-01-05' inclusive in sales transaction (column date). Return a single row with txn_count.

SQL Code:

```
-- Z3: Count transactions in date range

SELECT

   COUNT(*) AS txn_count

FROM salestransaction

WHERE tdate BETWEEN '2013-01-02' AND '2013-01-05';
```

OUTPUT:



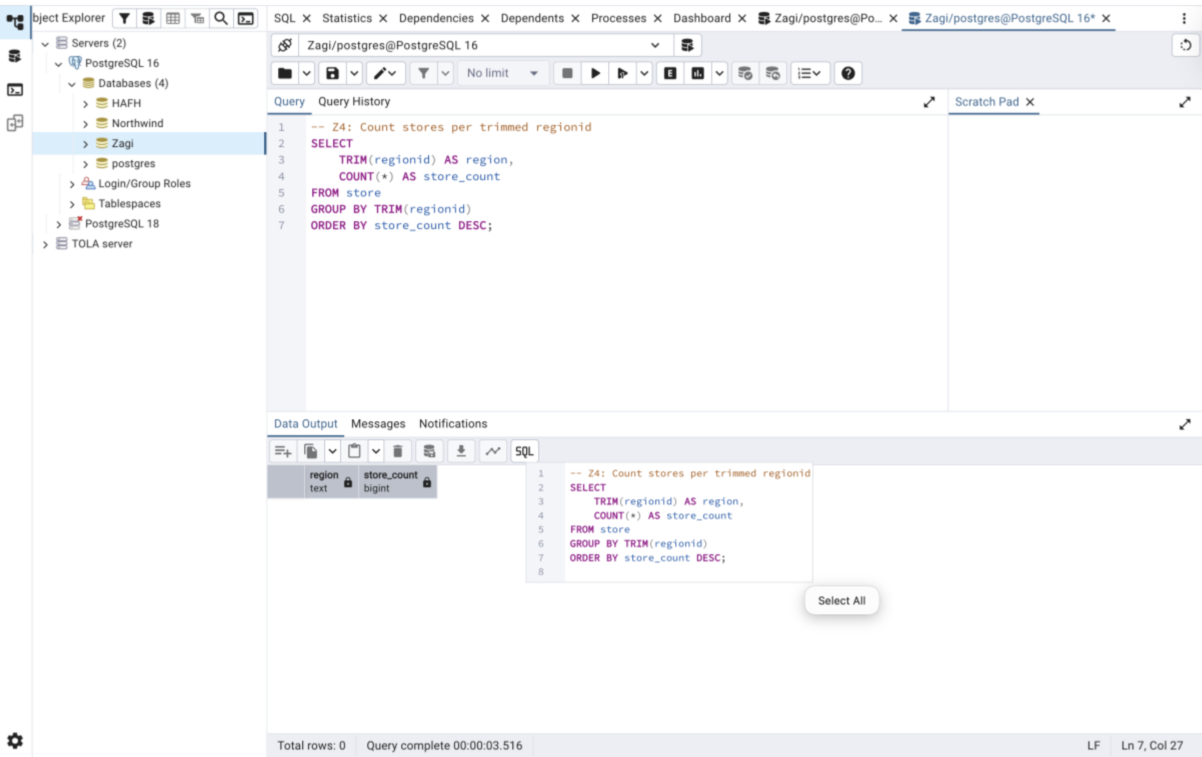
Z4 — Store Count Per Trimmed Region

Question.
For table store, return the number of stores per trimmed regionid as region, ordered by count desc—columns: region, store_count.

SQL Code:

```
-- Z4: Count stores per trimmed regionid

SELECT

   TRIM(regionid) AS region,

   COUNT(*) AS store_count

FROM store

GROUP BY TRIM(regionid)

ORDER BY store_count DESC;
```

OUTPUT:



Z5 — Top 3 Products by Quantity Sold

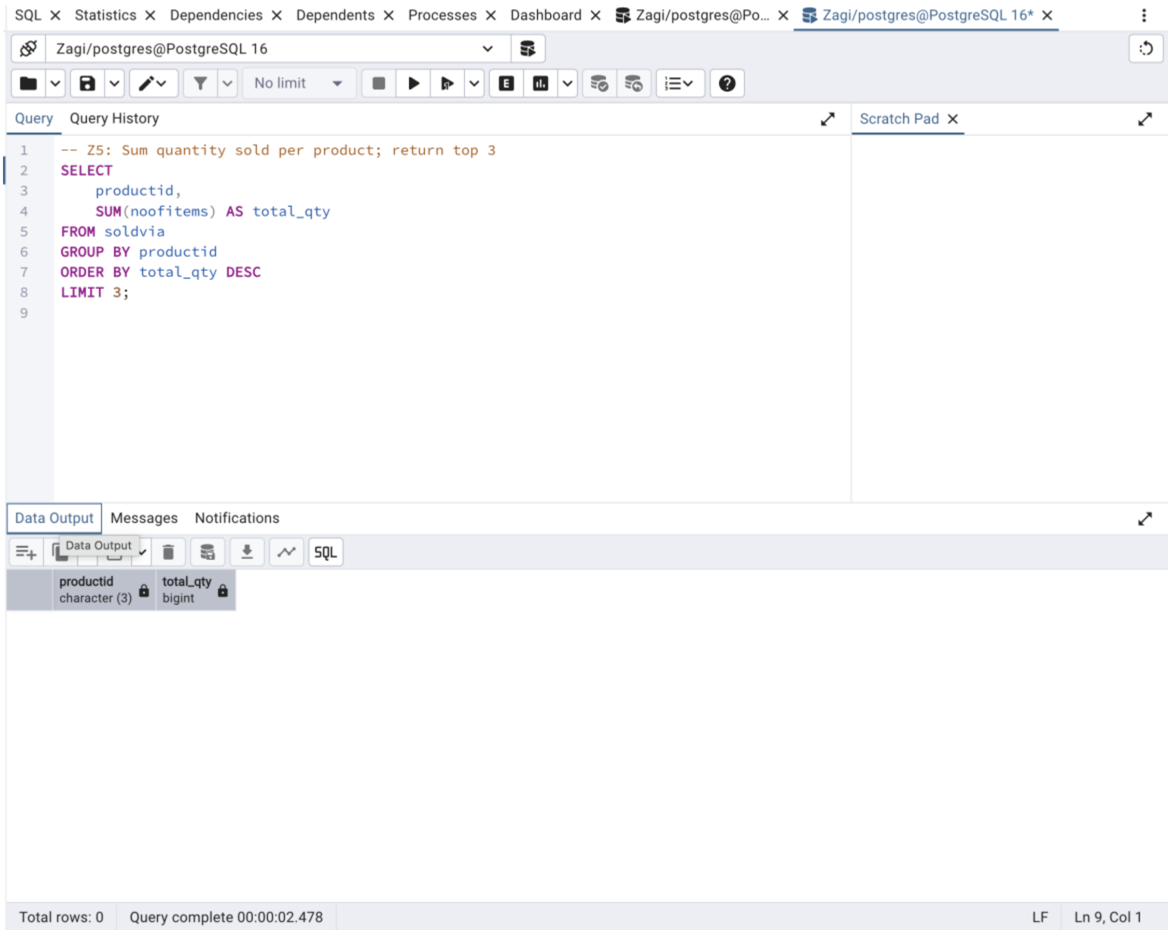
Question:

From soldvia, return the top 3 productid by total quantity sold (noofitems). Columns: productid, total_qty. Order by total_qty desc.

SQL Code:

```
-- Z5: Sum quantity sold per product; return top 3
SELECT
   productid,
   SUM(noofitems) AS total_qty
FROM soldvia
GROUP BY productid
ORDER BY total_qty DESC
LIMIT 3;
```

OUTPUT:



SECTION B — HAFH (5 QUERIES)

H1 — Salary stats + count of managers with bonus

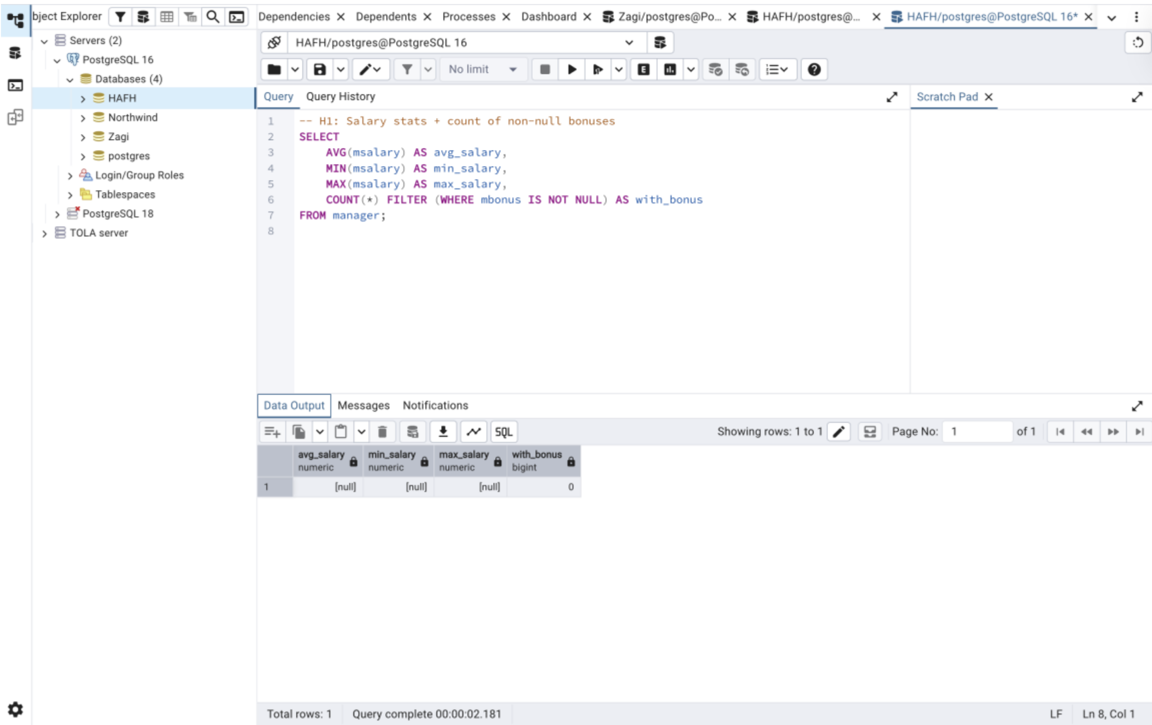
QUESTION:

Return AVG (msalary), MIN(msalary), MAX(msalary) as avg_salary, min_salary, max_salary, and COUNT (*) FILTER (WHERE mbonus IS NOT NULL) as with_bonus (or an equivalent.

SQL CODE:

```
-- H1: Salary stats + count of non-null bonuses

SELECT

    AVG(msalary) AS avg_salary,

    MIN(msalary) AS min_salary,

    MAX(msalary) AS max_salary,

    COUNT(*) FILTER (WHERE mbonus IS NOT NULL) AS with_bonus

FROM manager;
```

OUTPUT:



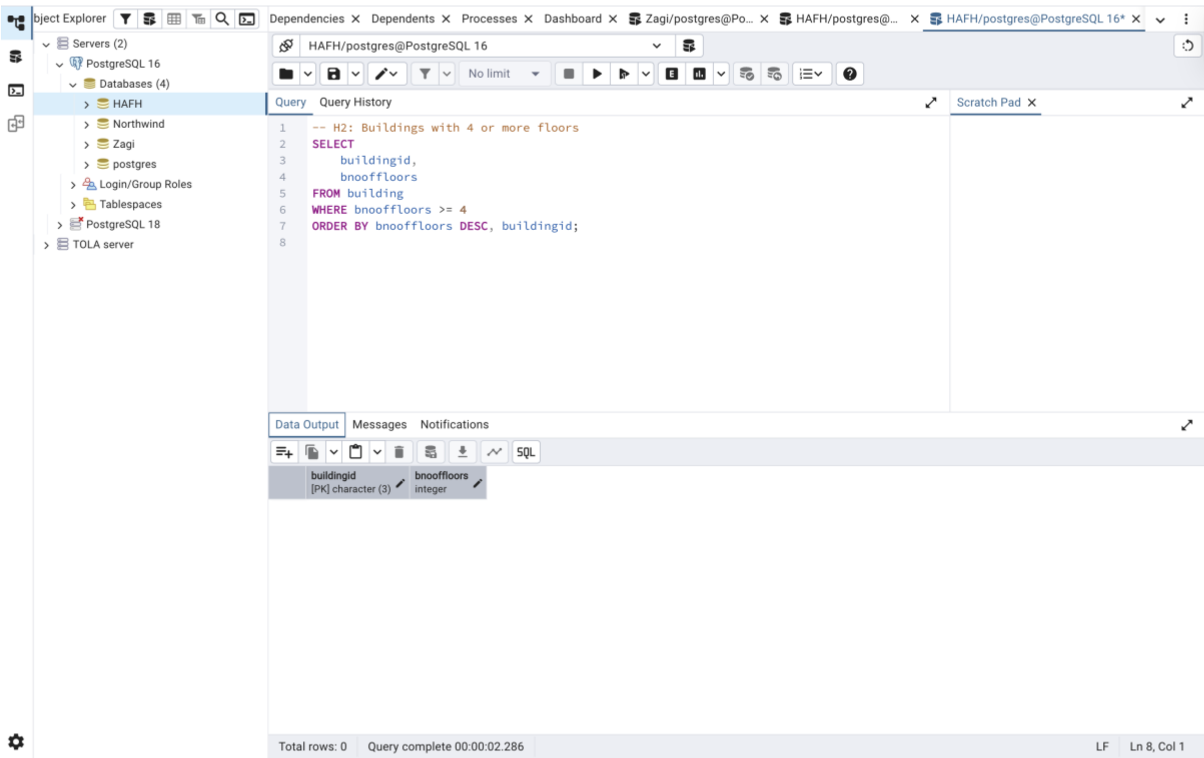
H2 — Buildings With ≥ 4 Floors

QUESTION:
List buildings with bnooffloors >= 4, ordered by bnooffloors desc then buildingid. Columns: buildingid, bnooffloors.

SQL CODE:

```
- H2: Buildings with 4 or more floors

SELECT

    buildingid,

    bnooffloors

FROM building

WHERE bnooffloors >= 4

ORDER BY bnooffloors DESC, buildingid;
```

OUTPUT:



H3 — Industries With ≥ 3 Corporate Clients

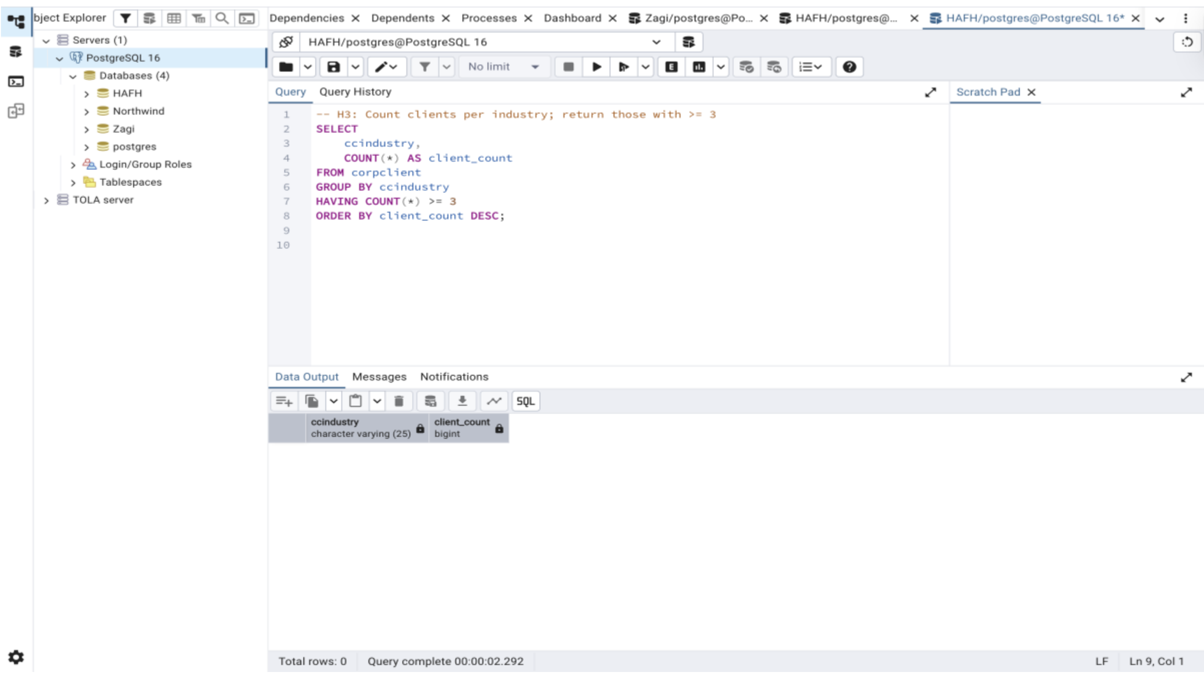
QUESTIONS:

Count clients per cc industry. Return industries with COUNT(*) >= 3, ordered by count desc. Columns: ccindustry, client_count.

SQL CODE:

```
-- H3: Count clients per industry; return those with >= 3

SELECT

    ccindustry,

    COUNT(*) AS client_count

FROM corpclient

GROUP BY ccindustry

HAVING COUNT(*) >= 3

ORDER BY client_count DESC;
```

OUTPUT:



H4 — Inspectors with ≥ 2 inspections scheduled in 2014

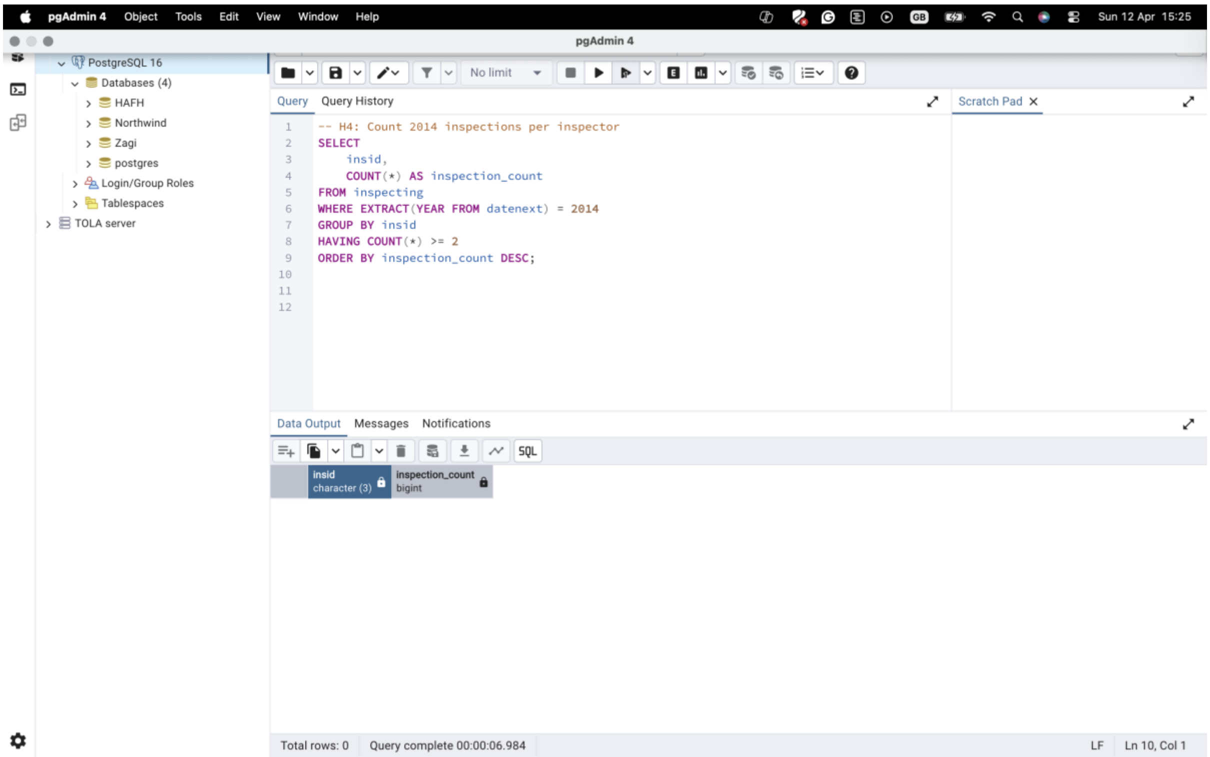
QUESTION:

For rows where datenext is in 2014, return inspectors with at least 2 scheduled inspections. Columns: insid, inspection_count, ordered by inspection_count desc.

SQL CODE:

```
 -- H4: Count 2014 inspections per inspector
SELECT
    insid,
    COUNT(*) AS inspection_count
FROM inspecting
WHERE EXTRACT(YEAR FROM datenext) = 2014
GROUP BY insid
HAVING COUNT(*) >= 2
ORDER BY inspection_count DESC;
```

OUTPUT:



H5 — Managers with ≥ 2 phone numbers

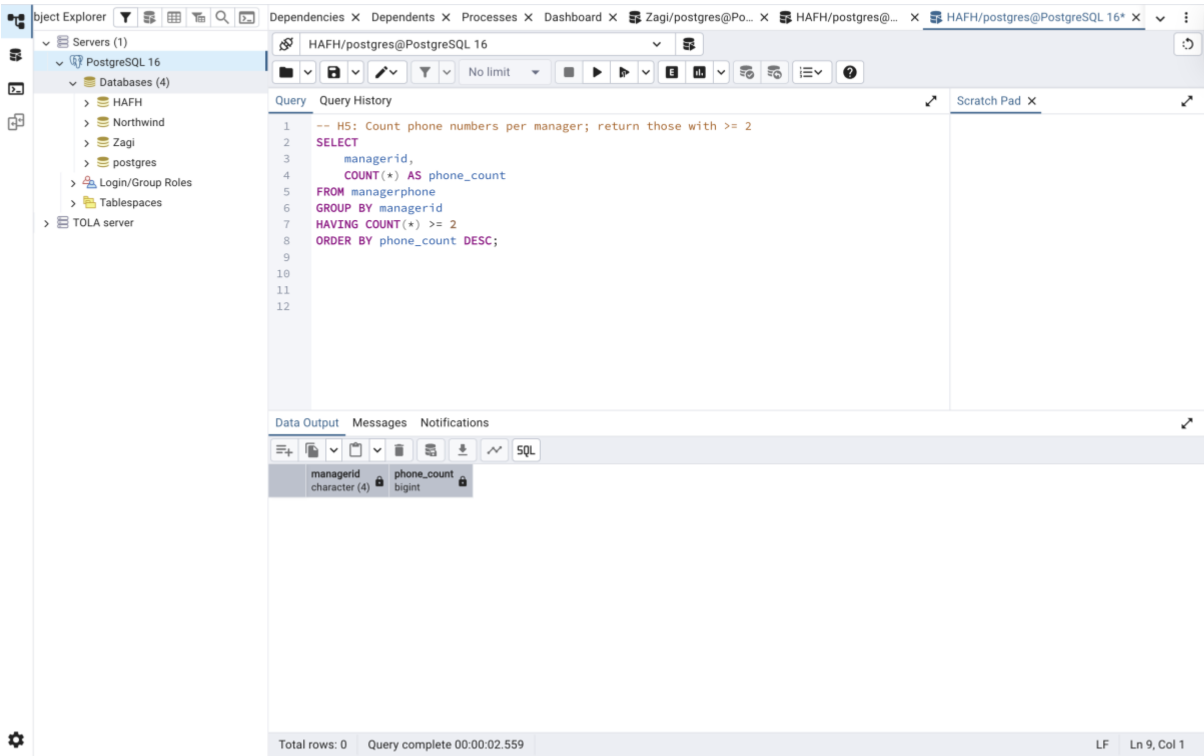
QUESTION:

Count phone numbers per manager and return managers with COUNT(*) >= 2. Columns: managerid, phone_count, ordered desc.

SQL CODE:

```
-- H5: Count phone numbers per manager; return those with >= 2
SELECT
    managerid,
    COUNT(*) AS phone_count
FROM managerphone
GROUP BY managerid
HAVING COUNT(*) >= 2
ORDER BY phone_count DESC;
```

OUTPUT:



SECTION C — NORTHWIND (5 QUERIES)

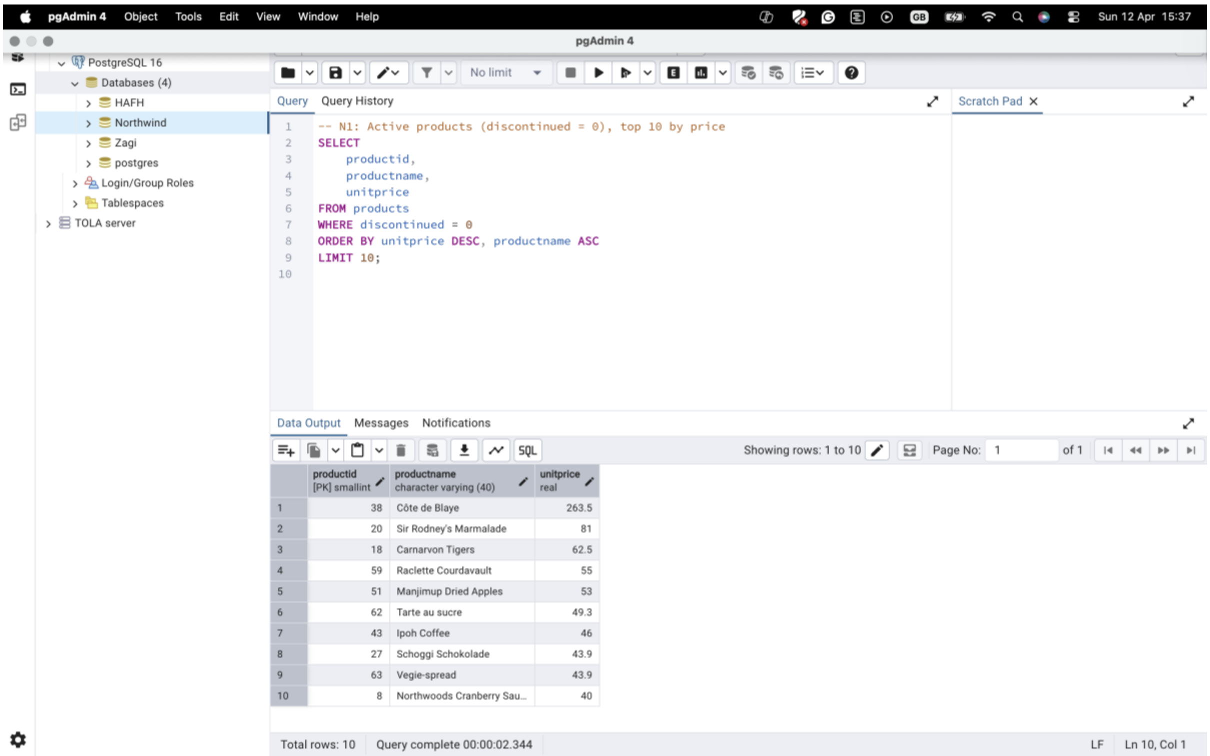
N1 — Top 10 most expensive active products

QUESTION:
“Which of our currently active products are the most expensive? Show the top 10 with their prices.”Analyst notes: Use products; filter active as discontinued = 0. Columns: productid, productname, unitprice. Sort by unitprice desc, then productname asc. Limit 10.

SQL CODE:

```
-- N1: Active products (discontinued = 0), top 10 by price

SELECT

    productid,

    productname,

    unitprice

FROM products

WHERE discontinued = 0

ORDER BY unitprice DESC, productname ASC

LIMIT 10;
```

OUTPUT:



N2 — Top 5 customer countries

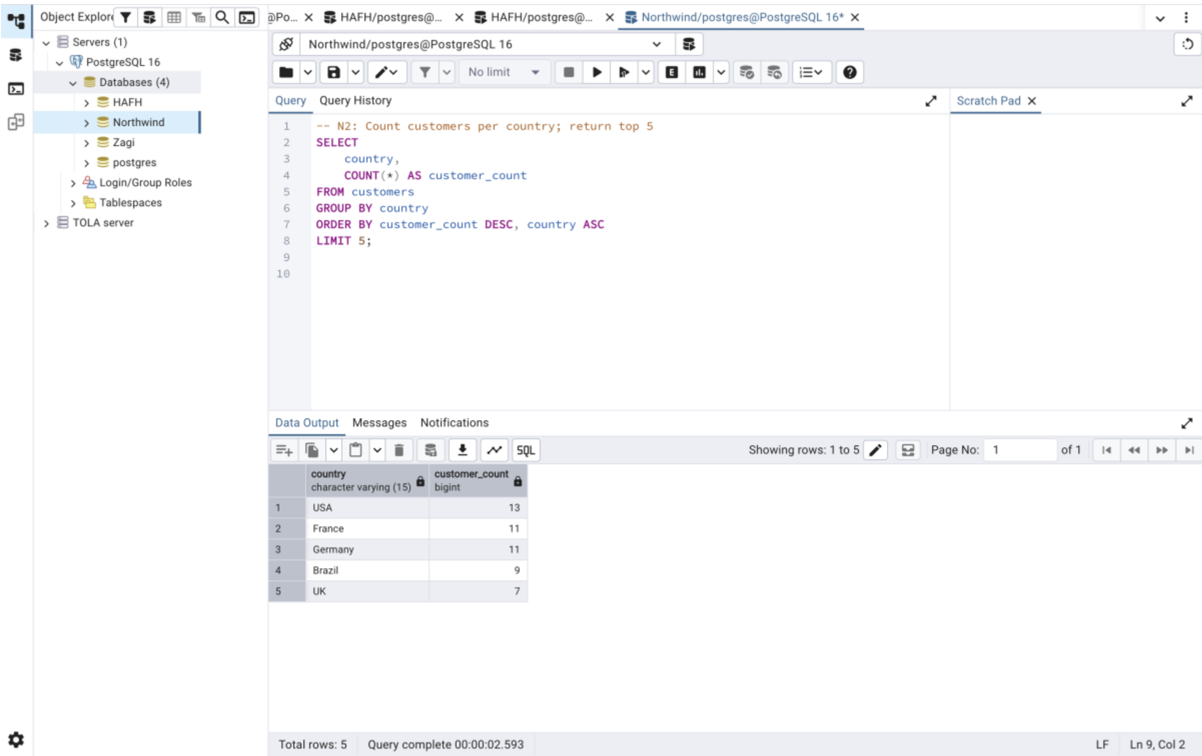
OUTPUT:

"Where are most of our customers located? Give me the top 5 countries by number of customers." Analyst notes: Use customers. Columns: country, customer_count. Sort by customer_count desc, then country asc. Limit 5.

SQL CODE:

```
-- N2: Count customers per country; return top 5
SELECT
    country,
    COUNT(*) AS customer_count
FROM customers
GROUP BY country
ORDER BY customer_count DESC, country ASC
LIMIT 5;
```

OUTPUT:



N3 — Number of orders received in 1996

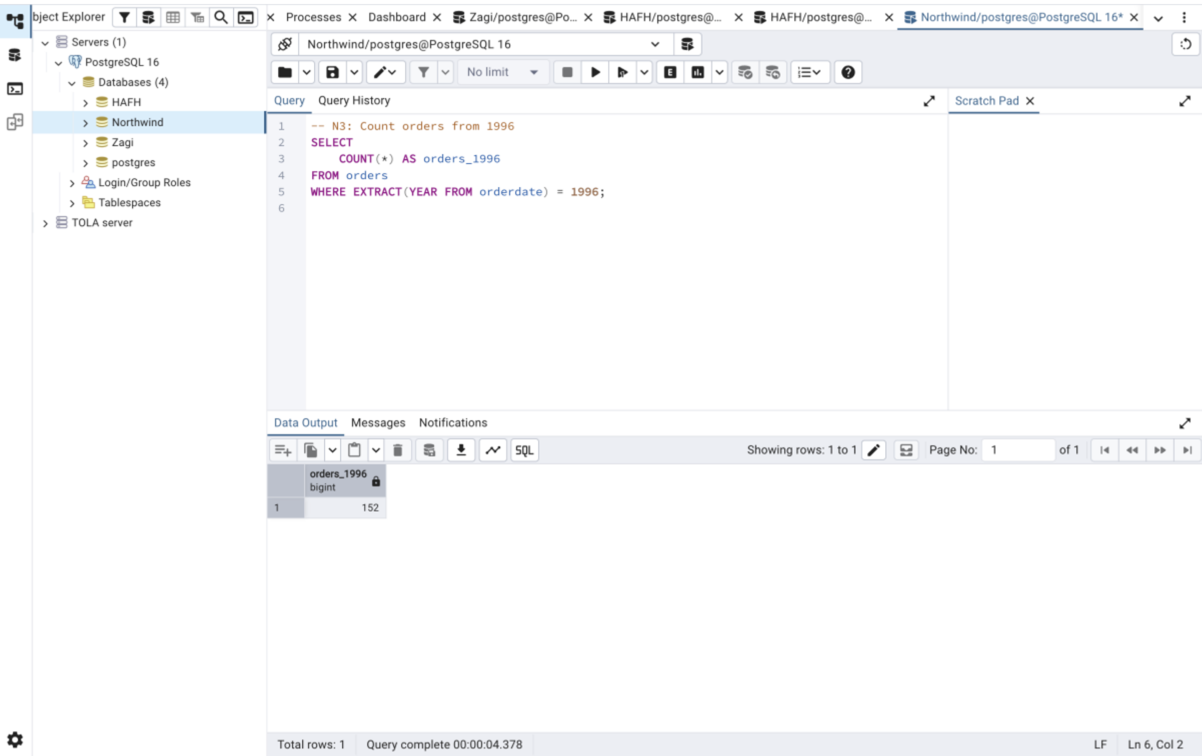
Question:

"How many orders did we receive in the year 1996?"Analyst notes: Use orders. Return a single row column named orders_1996.

SQL CODE:

```
-- N3: Count orders from 1996
SELECT
   COUNT(*) AS orders_1996
FROM orders
WHERE EXTRACT(YEAR FROM orderdate) = 1996;
```

OUTPUT:



N4 — Top 5 products by line-item revenue

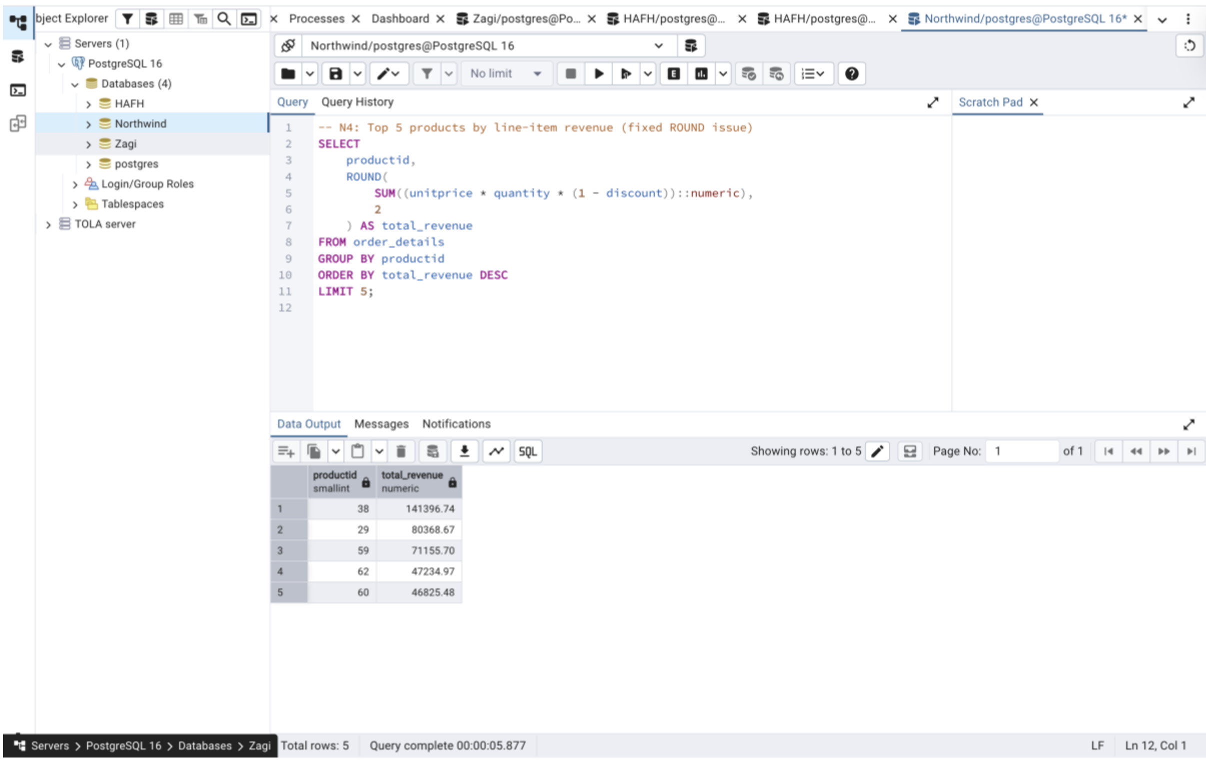
QUESTION:

“Which products drive the most line-item revenue? Give me the top 5 product IDs by revenue.”Analyst notes: Use order_details. Revenue per product = unitprice * quantity * (1 - discount) aggregated by productid. Columns: productid, total_revenue (round to 2 decimals). Sort desc. Limit 5.

SQL CODE:

```
-- N4: Top 5 products by line-item revenue (fixed ROUND issue)
SELECT
    productid,
    ROUND(
        SUM((unitprice * quantity * (1 - discount))::numeric),
        2
    ) AS total_revenue
FROM order_details
GROUP BY productid
ORDER BY total_revenue DESC
LIMIT 5;
```

OUTPUT:



N5 — Employees hired in 1993

QUESTION:
“Who joined the company in 1993? List their names and hire dates.”Analyst notes: Use employees—columns: employeeid, firstname, lastname, hiredate. Sort by lastname, then firstname.

SQL CODE:

```
-- N5: Employees hired in 1993
SELECT
    employeeid,
    firstname,
    lastname,
    hiredate
FROM employees
WHERE EXTRACT(YEAR FROM hiredate) = 1993
ORDER BY lastname, firstname;
```

OUTPUT: